

Análise de Resolução do Problema do Percurso em um Tabuleiro com Buscas e Heurísticas

Denilso Bernardo Nunes da Silva¹ Antonio Willian Silva Oliveira²

¹Bacharelado em Ciência da Computação
Universidade Federal do Ceará (UFC)
Ceará – CE – Brasil

denilsobn@alu.ufc.br williansilva@alu.ufc.br

Orientador: Prof. Samy Sá

Abstract. *This work proposes a comparative analysis of Hill Climbing, Greedy Search, and A* algorithms applied to a pathfinding problem on a 15x15 grid. Using Euclidean and Manhattan heuristics, four experiments evaluated the generated and visited states, solution cost, and success rate across varying obstacle densities. The results confirm theoretical properties: Hill Climbing struggles with local optima, Greedy Search is fast but suboptimal, and A* guarantees optimality at a higher computational cost. The study also demonstrates the impact of breaking heuristic admissibility in the Weighted A* approach.*

Resumo. *Este trabalho propõe uma análise comparativa dos algoritmos Subida de Encosta, Busca Gulosa e A* aplicados ao problema de percurso em um tabuleiro 15x15. Utilizando as heurísticas Euclidiana e Manhattan, quatro experimentos avaliaram os estados gerados e visitados, o custo da solução e a taxa de sucesso em diferentes densidades de obstáculos. Os resultados confirmam as propriedades teóricas: a Subida de Encosta sofre com ótimos locais, a Busca Gulosa é rápida mas subótima, e o A* garante otimalidade com maior custo computacional. O estudo também demonstra o impacto da quebra de admissibilidade heurística na variação do A* Ponderado.*

1. Introdução

O problema do percurso em um tabuleiro consiste em encontrar um caminho mínimo possível entre dois pontos em um tabuleiro de dimensões $n \times m$. Neste relatório, o tabuleiro tem dimensões fixas de 15×15 . Para todos os experimentos, utiliza-se como convenção que o estado inicial do problema começa no ponto (0, 0) (canto superior esquerdo) e termina no ponto (14, 14) (canto inferior direito).

Q0A: Validação de Entradas

É utilizado o algoritmo de Busca em Largura para validar a entrada. O algoritmo recebe a origem (0, 0) e o destino (14, 14). É garantido que não haja obstáculo na origem e no destino. Utiliza-se uma estrutura para colocar os estados gerados. O algoritmo segue com uma estrutura de fila, na qual explora todos os estados disponíveis (não visitados e livres), colocando na fila até chegar ao destino final. O algoritmo termina quando encontrar o destino final, retornando sucesso, ou quando a fila ficar vazia, retornando falha, já que todos os estados visitados são removidos da fila.

2. Experimentos

2.1. Experimento 1

Durante a execução desse primeiro experimento, foi notado que, ao utilizar o piso da heurística distância euclidiana na Subida de Encosta, o algoritmo finalizava no primeiro estado. O estado inicial era o ponto $(0, 0)$ e o final, $(14, 14)$. A distância euclidiana d desses dois pontos era de $d = \lfloor \sqrt{14^2 + 14^2} \rfloor = 19$. A distância dos dois pontos adjacentes até o estado final era de $d_1 = \lfloor \sqrt{13^2 + 14^2} \rfloor = 19$ e $d_2 = \lfloor \sqrt{14^2 + 13^2} \rfloor = 19$. Como essas três distâncias são iguais, o algoritmo permanecia no primeiro estado, já que para passar para o próximo estado o valor deveria ser menor que o estado atual. Se utilizasse o menor ou igual ($\leq$) como uma pequena variação da subida de encosta, poderia entrar em um caso em que o próximo estado poderia ser um estado anterior, já que vimos que pontos podem ter valores iguais. Como o uso do piso era apenas uma recomendação para fins de complexidade de implementação, manteve-se o uso da distância euclidiana sem o piso.

Decidimos agregar os resultados do experimento 1 para uma melhor análise. Esses resultados se encontram na tabela 1.

Tabela 1. Médias de Desempenho

Densidade	Algoritmo	Média Gerados	Média Visitados	Média Custo (Soluções)
0%	SdE Gulosa Det. (Euclidiana)	41.0	29.0	126.4
	SdE Gulosa Det. (Manhattan)	41.0	29.0	126.4
	SdE Maior Aclive (Euclidiana)	108.0	29.0	123.1
	SdE Maior Aclive (Manhattan)	82.0	29.0	126.4
	SdE Gulosa Estoc. (Euclidiana)	104.2	29.0	125.1
	SdE Gulosa Estoc. (Manhattan)	103.2	29.0	126.7

Q1A

Observando os dados brutos e a tabela 1, a variação SdE Maior Aclive com distância Euclidiana apresentou o menor custo de solução.

Isso ocorreu devido à própria natureza do algoritmo de Maior Aclive. O Maior Aclive avalia todos os vizinhos disponíveis antes de decidir. Quando combinado com a métrica Euclidiana, que calcula a distância em linha reta, o algoritmo consegue identificar a rota localmente mais direta a cada passo em um cenário sem obstáculos, resultando no caminho mais otimizado.

Q1B

Não houve variação com desempenho superior evidente. As três variações utilizando a Distância Manhattan apresentaram custos semelhantes.

Isso acontece porque, em um ambiente totalmente sem obstáculos, a métrica de Manhattan calcula a distância estritamente através da soma das diferenças nos eixos X e Y. Consequentemente, múltiplos caminhos adjacentes na direção do objetivo apresentarão exatamente o mesmo ganho heurístico. Isso faz com que várias opções de movimento pareçam igualmente boas. Como o algoritmo não consegue desempatar-las de forma que gere um ganho real, todas as variações acabam produzindo caminhos de custo muito semelhante.

Q1C

A Distância Euclidiana com maior aative, na maioria das vezes, produziu caminhos mais curtos em comparação com a Manhattan. Isso mostra que a distância euclidiana fornece um direcionamento ao agente de forma mais eficiente para o alvo.

Em contrapartida, também observamos uma diferença no esforço computacional, a versão Maior Aactive gerou 108 estados com a heurística Euclidiana, enquanto com a Manhattan gerou apenas 82. A métrica euclidiana tentará seu caminho pelo meio do tabuleiro, gerando mais estados, já que todas as direções estão livres. Já a Manhattan segue pelo canto do tabuleiro, onde tem menos direções para seguir, gerando menos estados.

2.2. Experimento 2

Decidimos agregar os resultados do experimento 2 para uma melhor análise. Esses resultados se encontram na tabela 2.

Tabela 2. Desempenho da Busca Gulosa e A* sem obstáculos (0% Densidade)

Densidade	Algoritmo	Média Gerados	Média Visitados	Média Custo (Soluções)	Taxa de Sucesso
0%	Busca Gulosa (Euclidiana)	61.1	29.0	115.4	100.0%
	Busca Gulosa (Manhattan)	64.1	29.0	113.8	100.0%
	A* (Euclidiana)	900.0	226.0	103.9	100.0%
	A* (Manhattan)	900.0	226.0	103.9	100.0%

Q2A

De acordo com a média de custo e os custos de todos os experimentos, o A* se mostrou com custo inferior ao custo da Busca Gulosa. Isso é esperado porque a heurística utilizada no A* é admissível e sempre subestima o custo real, garantindo a otimalidade do algoritmo. Já a Busca Gulosa, como considera apenas a heurística para mudar seu estado, pode seguir por caminhos de maiores custos.

Q2B

Tabela 3. Resultados dos Experimentos para o Mapa 4

Mapa	Algoritmo	Gerados	Visitados	Custo	Sucesso
4	Busca Gulosa (Euclidiana)	58	29	116	Sim
4	Busca Gulosa (Manhattan)	62	29	119	Sim
4	A* (Euclidiana)	896	225	108	Sim
4	A* (Manhattan)	896	225	108	Sim

Teoricamente, como o A^* considera o custo acumulado e o tabuleiro tem custos variados, ao passar do algoritmo, o custo acumulado começa a dominar na função. Dessa forma, o algoritmo começa a funcionar semelhantemente ao algoritmo de Dijkstra. No exemplo da tabela 3, o A^* visitou 225 estados, ou seja, o tabuleiro inteiro. Na Busca Gulosa, foi possível observar que ela visitou significativamente menos estados do que no A^* . Como ela considera a heurística para mudar de estado, ela seguirá diretamente para o objetivo, mesmo que o custo aumente. Isso é observado na tabela 3, onde a Busca Gulosa mostra custos superiores ao do A^* .

Q2C

No A^* , não foi notada uma diferença significativa. Isso é justificado pelo ponto discutido anteriormente. Na Busca Gulosa, a distância euclidiana gerou uma quantidade inferior de estados do que a distância Manhattan na maioria dos casos. Isso é justificável porque, com a distância euclidiana, segue-se o caminho mais próximo da linha reta, fazendo com que a quantidade de estados gerados nas direções ortogonais seja menor pela quantidade de estados visitados anteriormente. A distância Manhattan se depara com vários estados que dão empates, já que em alguns casos seguir pela direita ou para baixo tem o mesmo custo, fazendo com que gere mais estados. Os custos da Busca Gulosa foram bastante variados, mesmo que a média da distância Manhattan tenha sido maior, os resultados de cada mapa mostram que o menor custo se alterna entre as heurísticas.

2.3. Experimento 3

Os resultados agregados se encontram na tabela 4.

Tabela 4. Médias de Desempenho e Taxa de Sucesso por Heurística

Densidade	Algoritmo	Média Gerados	Média Visitados	Média Custo (Soluções)	Taxa de Sucesso
20%	A* (h1 = 1x D_E)	696.4	175.1	116.0	100.0%
	A* (h2 = 3x D_E)	652.0	164.0	116.0	100.0%
	A* (h3 = 6x D_E)	254.0	64.5	117.2	100.0%
	A* (h4 = 1x D_M)	692.8	174.2	116.0	100.0%
	A* (h5 = 3x D_M)	598.4	150.6	116.0	100.0%
	A* (h6 = 6x D_M)	164.6	42.1	122.3	100.0%

Q3A

Percebe-se que tanto na heurística da distância euclidiana quanto na de Manhattan, a quantidade de estados gerados e visitados diminui à medida que o fator de multiplicação aumenta. Em contraposição, a Média Custo aumenta nas soluções. No fator 6, a média dos estados gerados e visitados diminui drasticamente.

Q3B

Os fatores multiplicativos afetaram diretamente as respostas finais do algoritmo. Observando os dados, com os fatores 1x e 3x, o A* encontrou o caminho de menor custo para ambas as heurísticas. No entanto, ao aplicarmos o fator 6x, o custo subiu para 117,2 (Euclidiana) e 122,3 (Manhattan), indicando que o algoritmo encontrou caminhos que não eram ótimos.

Isso acontece porque multiplicar a heurística por uma constante positiva transforma o A* tradicional em uma variante conhecida como A* Ponderado, citado no livro da disciplina, na qual a função de avaliação passa a ser $f(n) = g(n) + W \times h(n)$. À medida que o fator aumenta, o valor da heurística $h(n)$ passa a dominar o custo real percorrido $g(n)$, transformando-se em um valor superestimado.

Com o fator 6x, o algoritmo se torna muito direto para chegar ao objetivo. Se houver um obstáculo no caminho direto, o algoritmo com $W = 6$ preferirá contorná-lo rapidamente por um caminho visivelmente mais custoso a longo prazo do que retroceder alguns passos para pegar uma rota globalmente melhor, pois retroceder faria o valor de $6 \times h(n)$ aumentar, fazendo o algoritmo não explorar essa rota. Ele passa a se comportar de forma muito semelhante à Busca Gulosa, aumentando o valor da resposta final em troca de expansões mais rápidas.

Q3C

Os resultados observados confirmam a teoria de admissibilidade de heurísticas.

Teoricamente, uma heurística é considerada admissível se ela nunca superestima

o custo real para atingir o objetivo, ou seja, $h(n) \leq h^*(n)$. Quando a heurística é admissível, o algoritmo A^* tem a propriedade teórica de garantir a otimalidade.

- **Fator 1x:** Representa a heurística. Sendo as distâncias Euclidiana e Manhattan admissíveis para esse domínio, o A^* garante a otimalidade, encontrando o menor custo. Porém, por ser um limite inferior seguro, o algoritmo precisa explorar mais opções.
- **Fator 3x:** Multiplicar a heurística por 3 faz com que ela superestime o custo real, perdendo a sua propriedade de admissibilidade teórica. Na prática observada, o algoritmo ainda encontrou a solução ótima com menos esforço computacional, mas a garantia matemática de otimalidade já não existia.
- **Fator 6x:** Uma superestimação ainda maior. A admissibilidade é totalmente quebrada. Conforme a teoria prevê, ao utilizar uma heurística fortemente inadmissível, o A^* perde a garantia de otimalidade. Os dados refletem isso: a quebra da admissibilidade permitiu que o algoritmo gerasse e visitasse menos estados, mas retornou soluções que não eram ótimas.

2.4. Experimento 4

Tabela 5. Médias de Desempenho e Taxa de Sucesso por Densidade e Algoritmo

Densidade	Algoritmo	Média Gerados	Média Visitados	Média Custo (Soluções)	Taxa de Sucesso
10%	SdE Maior Aclive	56.4	18.9	125.5	20.0%
	Busca Gulosa	61.6	30.8	115.7	100.0%
	A^*	820.0	206.0	107.6	100.0%
20%	SdE Maior Aclive	42.2	15.1	128.0	5.0%
	Busca Gulosa	57.6	33.1	121.4	100.0%
	A^*	709.0	178.2	114.2	100.0%
30%	SdE Maior Aclive	32.0	12.2	0.0	0.0%
	Busca Gulosa	60.5	41.4	128.0	100.0%
	A^*	571.8	143.9	119.8	100.0%
40%	SdE Maior Aclive	21.3	8.8	0.0	0.0%
	Busca Gulosa	57.2	42.4	134.2	100.0%
	A^*	387.8	98.0	129.2	100.0%

Q4A

Para uma análise completa, dividiremos a discussão para cada métrica gerada da tabela 5.

Taxa de Sucesso

É possível notar que a taxa de sucesso da Subida de Encosta com o maior Aclive decresceu à medida que a densidade de obstáculos aumentava, chegando a ser zero nas duas últimas

densidades. Isso se deve ao fato de que a Subida de Encosta não tem memória, então existe a possibilidade de esse algoritmo entrar em um estado sem saída. À medida que os obstáculos aumentam, maior a possibilidade de se entrar em um estado sem saída.

Como o tabuleiro gerado sempre tem solução por causa da verificação com uma busca em largura, a Busca Gulosa e o A^* sempre encontram uma solução, explicando sua taxa de sucesso absoluta.

Estados Gerados e Visitados

O algoritmo A^* apresenta um número significativamente maior de estados gerados e visitados em comparação aos demais, refletindo sua natureza exploratória para garantir a solução ótima. À medida que a densidade de obstáculos aumenta, as médias de estados gerados e visitados do A^* diminuem (de 820,0 para 387,8 estados gerados). Isso ocorre porque a maior presença de obstáculos reduz o fator de ramificação do problema, limitando os caminhos e ramificações possíveis que o A^* precisa colocar em sua fronteira.

A Busca Gulosa mantém uma média de estados gerados relativamente estável, mas seus estados visitados aumentam com a densidade. Isso pode indicar que, ao se deparar com obstáculos que bloqueiam a rota apontada pela sua heurística, o algoritmo contorna aquele obstáculo, visitando mais nós para encontrar saídas. Esse padrão foi observado na visualização dos caminhos feitos no algoritmo.

Já a Subida de Encosta com Maior Aclive apresenta uma queda constante nos estados gerados e visitados. Isso indica que o algoritmo falha cada vez mais rápido. Ao atingir estados sem saída gerados pela alta concentração de obstáculos, o algoritmo para a execução.

Custo da Solução

Avaliando a coluna Média Custo, percebe-se que o A^* obteve os menores custos em todas as densidades. A Busca Gulosa apresentou custos mais elevados. Em todos os casos que obtiveram sucesso, o custo das soluções aumenta com a elevação da densidade de obstáculos, o que era esperado, visto que o agente é forçado a realizar percursos mais longos com mais desvios para chegar ao objetivo.

Q4B

O algoritmo mais afetado negativamente foi a Subida de Encosta com Maior Aclive.

Os dados sustentam essa conclusão por meio da métrica Taxa de Sucesso. Na densidade mais baixa, o algoritmo já apresentava um desempenho abaixo do esperado. À medida que a densidade aumentou para 20%, a taxa caiu para 5% e, a partir de 30%, falhou em 100% das execuções. O aumento de densidade eleva a ocorrência de estados sem saída, fazendo o algoritmo de busca sem memória falhar.

Vale ressaltar também que o A^* foi o mais afetado da perspectiva do esforço computacional. A redução de opções de movimentação devido ao aumento da densidade de

obstáculos reduziu a média de estados gerados do A^* em mais da metade.

Q4C

Sim, os resultados convergem para as características teóricas discutidas para cada um destes mecanismos de busca:

- **Subida de Encosta com Maior Aclive:** Teoricamente, é caracterizado como um algoritmo de busca local incompleto, que sofre para lidar com situações em que nenhum estado vizinho melhora o valor da heurística. Os dados refletem exatamente essa limitação de funcionamento, resultando em sucessivas falhas que mostram taxas de sucesso de 0% em ambientes densos.
- **Busca Gulosa:** O mecanismo de funcionamento da Busca Gulosa foca em expandir o nó que parece estar mais próximo do objetivo segundo a heurística. Embora chegue rapidamente ao objetivo, ele não garante a solução ótima. Os dados provam essa característica: o algoritmo conseguiu 100% de sucesso, porém entregou médias de custo sempre superiores ao do A^* . Ele sofre desvios e vai para estados que inicialmente parecem bons, mas que a longo prazo não são os mais eficientes.
- **A^* :** O A^* é conhecido teoricamente por ser completo e ótimo, utilizando uma heurística admissível. Os dados estão alinhados a isso, mantendo 100% de taxa de sucesso e retornando o menor custo de solução dentre as técnicas avaliadas. Como desvantagem teórica, o A^* exige alto custo espacial e de tempo. Isso também é reproduzido nos dados, visto que o número de estados gerados e visitados do A^* é o maior entre todos os algoritmos.